3.1 Theoretical Background

3.1.1 Hash Value/Digital fingerprinting:

Digital fingerprinting is based on the use of a mathematical function to produce a numerical value where the function takes an arbitrary length of data as its input and outputs a numerical value of a specified length; 128 bits (16 bytes), 256 bits (32 bytes) and 512 bits (64 bytes) are typical lengths. The output value of such a function is generally referred to as a hash value or simply hash. In order to be useful, the function used to produce a digital fingerprint must be quick to use but in order to be trustworthy for authentication purposes, it must also meet certain security requirements. For this reason, digital fingerprinting uses **cryptographic hash functions**.

Two of the most commonly encountered functions are **MD5** and the **SHA** family of functions.

3.1.1.1 MD5 (Message-Digest algorithm):

In cryptography, **MD5 (Message-Digest algorithm 5)** is a widely used cryptographic hash function with a **128-bit hash value**. As an Internet standard (RFC 1321), MD5 has been employed in a wide variety of security applications, and is also commonly used to check the integrity of files. An MD5 hash is typically expressed as a **32 digit hexadecimal number**. MD5 is a strengthened version of MD4. Like MD4, the MD5 hash was invented by Professor Ronald Rivest of MIT. Also, MD5 was obviously used as the model for SHA-1, since they share many common features. MD5 and SHA-1 are the two most widely used hash algorithms today, but use of MD5 will certainly decline over time, since it is now considered broken.

3.1.1.2 SHA (Secure Hash Algorithm):

Secure Hash Algorithm (SHA) is a family of cryptographic hash functions developed by the US National Security agency (NSA) and published as standards by the US National Institute of Science and Technology (NIST). It is the required algorithm for secure applications used by US government agencies. An important feature of the SHA algorithms is that they implement an avalanche effect which means that a minor change to the input cascades into a major change in the output value. The first version, **SHA-1** produces a **160-bit value** and was released in 1993; it was, however, withdrawn shortly after publication due to an undisclosed vulnerability and a modified version was released two years later. In 2002, NIST published the **SHA-2** family of functions. Unlike SHA-1 with hash size fixed at 160 bits, SHA-2 is offered in six versions producing a range of output sizes from **224 to 512 bits** of which the most widely used are **SHA-256** and **SHA-512**.

3.1.2 What are the benefits of Hashing?

One main use of hashing is to **compare two files for equality**. Without opening two document files to compare them word-for-word, the calculated hash values of these files will allow the owner to know immediately if they are different.

Hashing is also used to verify the integrity of a file after it has been transferred from one place to another, typically in a **file backup program** like **SyncBack**. To ensure the transferred file is not corrupted, a user can compare the hash value of both files. If they are the **same**, then the transferred file is an identical copy.

3.2 Instructions

How to verify a file or a set of files?

3.2.1 Using the **PowerShell Get-FileHash** Cmdlet in Windows

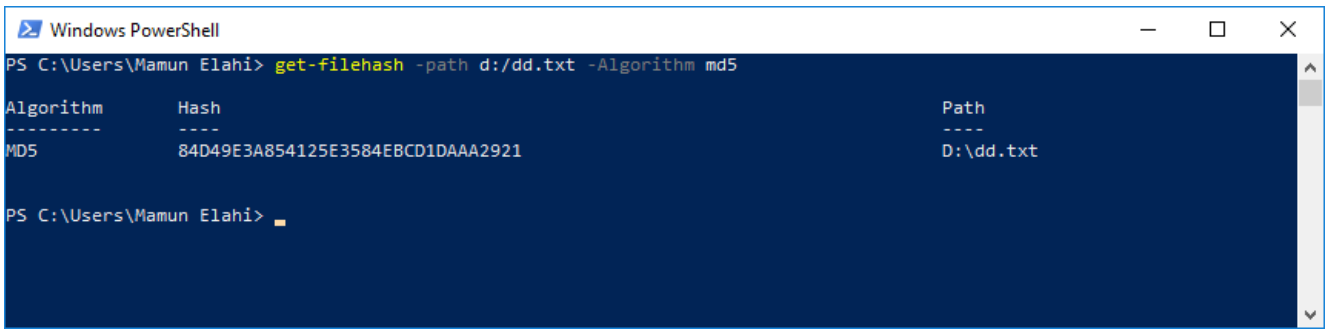
Get-FileHash PowerShell cmdlet is a command-line tool to create and verify hashes. Refer to the syntax below to know the available parameters. Use the **Path** parameter to specify **a target file**, and the **Algorithm** parameter to indicate which **hash algorithm to use**.

```
Get-FileHash -Path [path to file] -Algorithm [Hash Algorithm]
```

Some of the available hash algorithms for Get-FileHash are listed below:

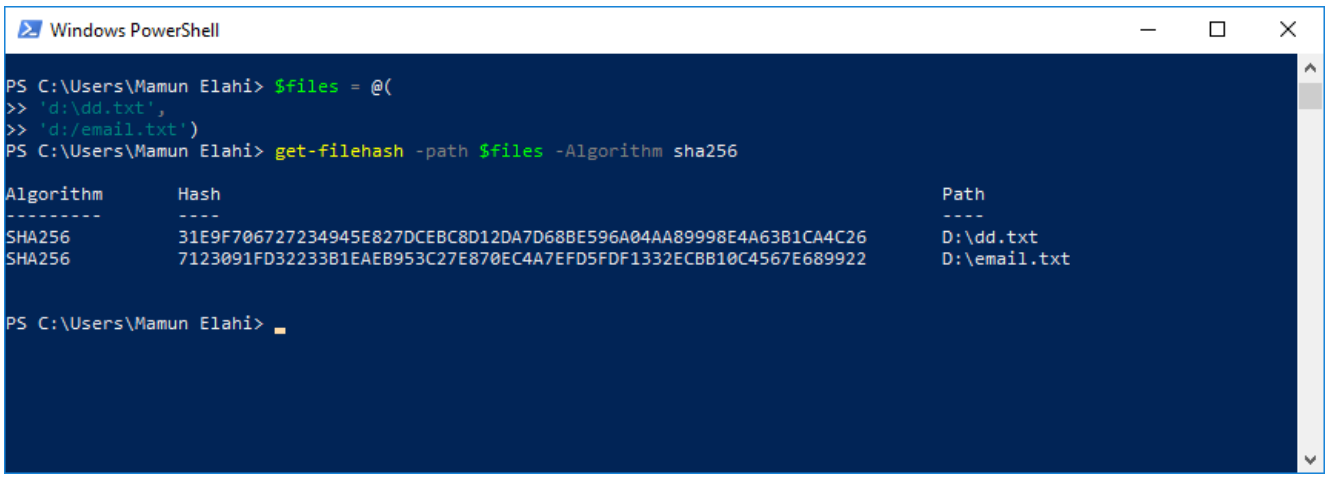
SHA1 / SHA256 / SHA384 / SHA512 / MD5

As you can see in the screenshot below, the Get-FileHash command computed the file's **MD5** hash as specified with the -Algorithm D5 parameter. If you want to compute the file's checksum using different algorithms, you only need to change the Algorithm parameter value.



Generating Hashes For Multiple Files Simultaneously

With Get-FileHash, it is possible to generate the checksum of **multiple files** at the same time. The example code below stores the list of file paths within the **$files** variable. Next, the Get-FileHash consumes the **$files** variable to calculate the **SHA-256** checksum for each file, as shown in the screenshot below:



3.2.2 Using **Third-Party Tools** to Create File Checksums in Windows

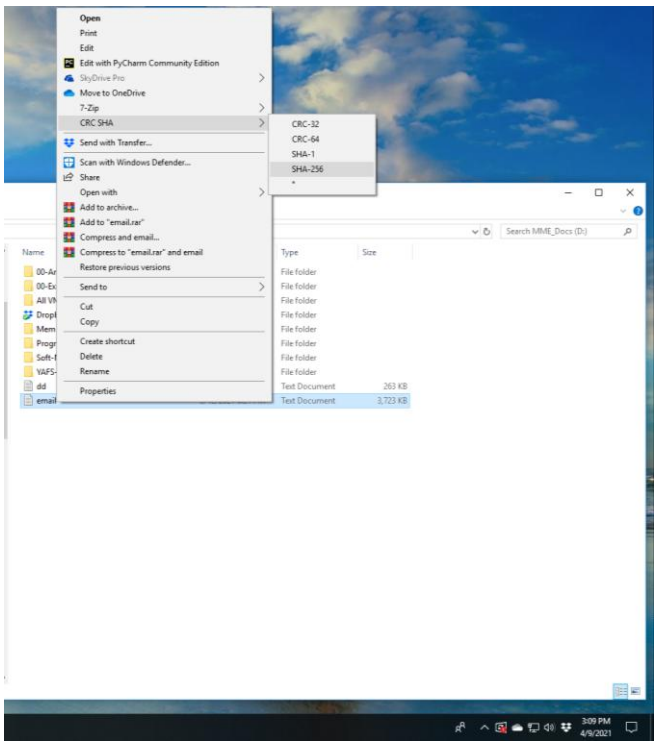
If for some reason, you find that the built-in tools to generate file checksums in Windows are not enough, there are third-party tools available. Not all third-party tools are free, but a few free and popular utilities are shown below.

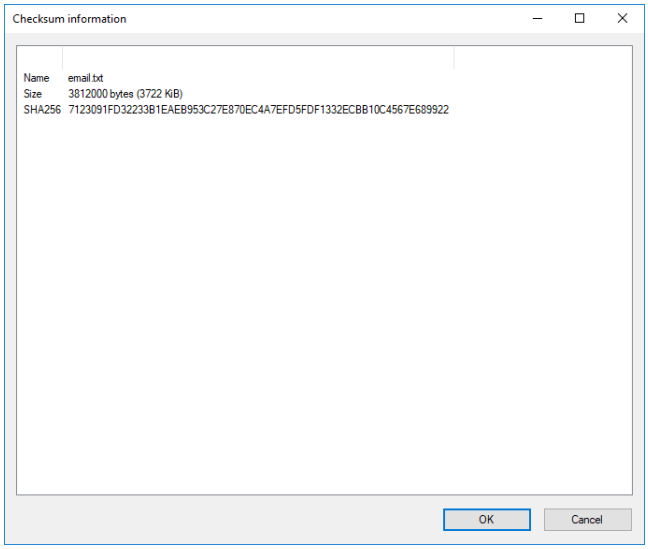
3.2.2.1 7-Zip

7-zip is a popular and free file compression tool that is capable of generating a file's hash value. Installing 7-zip on your computer **automatically adds a context menu option to generate the checksum** of a file. To get the hash value of a file, follow these steps:

1. Right-click on the file to generate the checksum.
2. Select the **CRC SHA menu option** to list the available hash algorithms.
3. Finally, click on the hash algorithm you wish to use.

In the example below, the **asterisk (*) option** displays the file's hash value using all available algorithms.

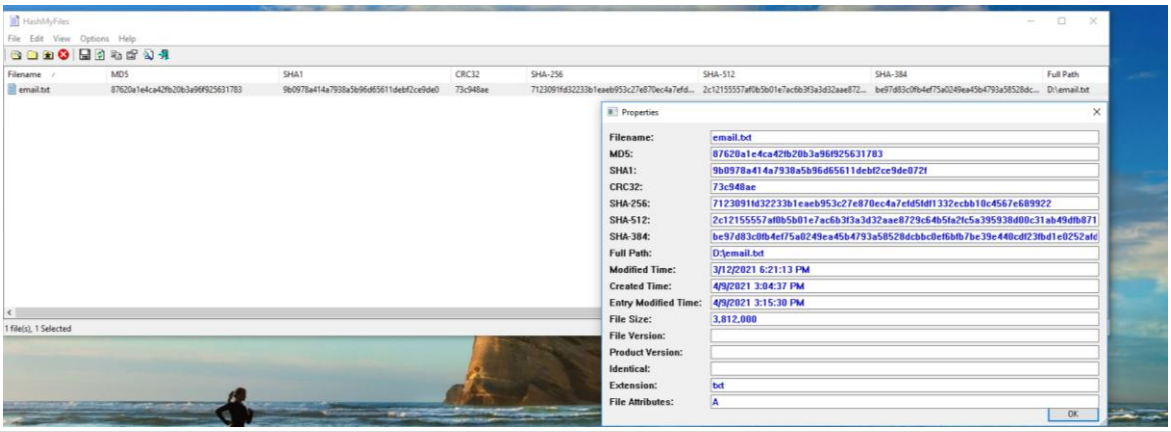




3.2.2.2 HashMyFiles

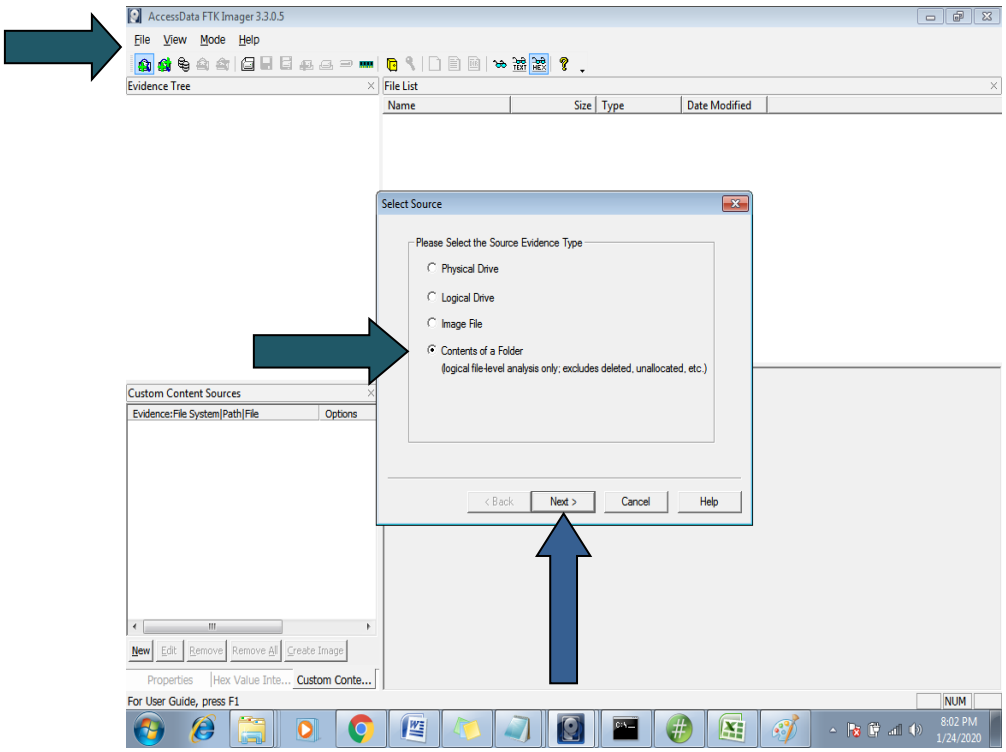
HashMyFiles can create the checksum or hash values of a single file, multiple files, or entire directories using drag and drop. Launch the HashMyFiles program and **drag the files or folders** to generate a hash for into the HashMyFiles main window. As shown below, the **main window** displays the **files and associated hashes** in a **table format**, with additional information available in the **properties** window.

HashMyFiles can **export the hash or checksum values to an HTML report or a tab-delimited file**. This export feature is useful when you want to keep a record of the file checksum values.

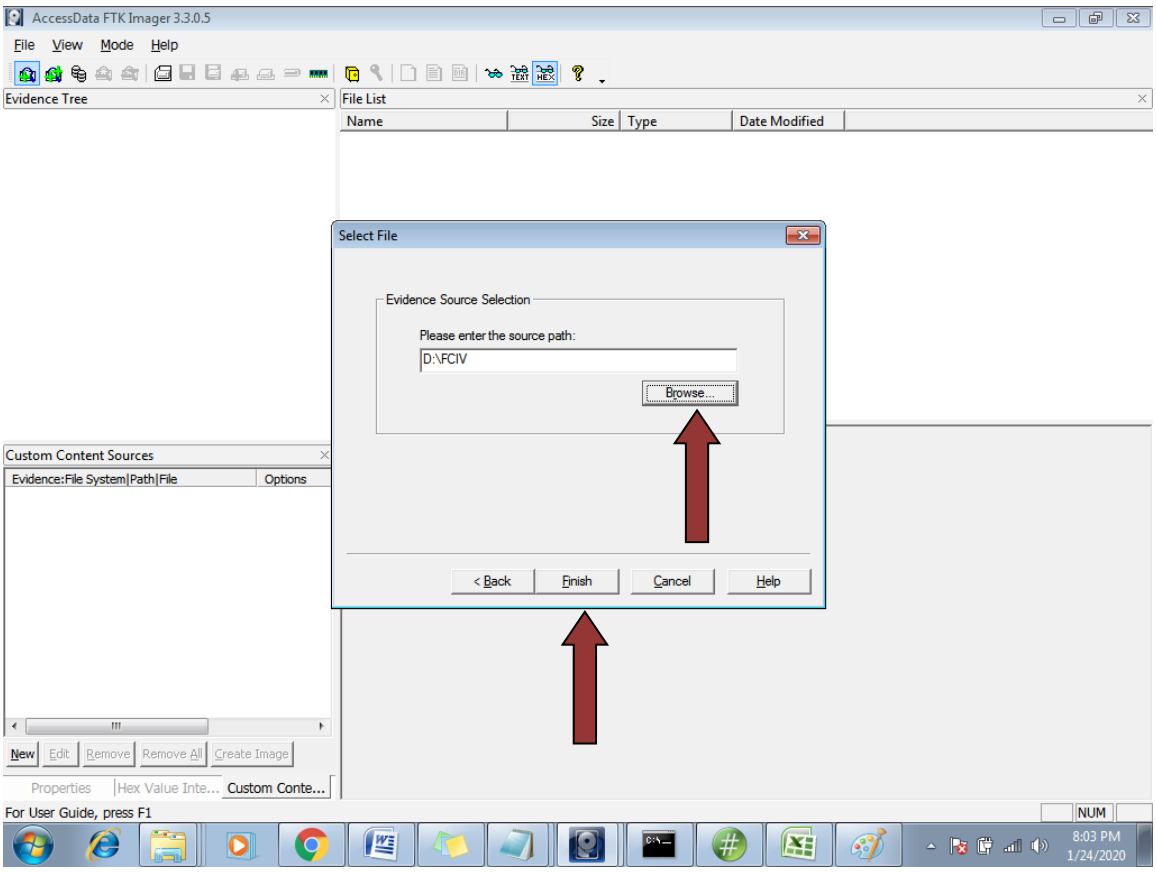


3.2.3 Using **FTK Imager Tool**

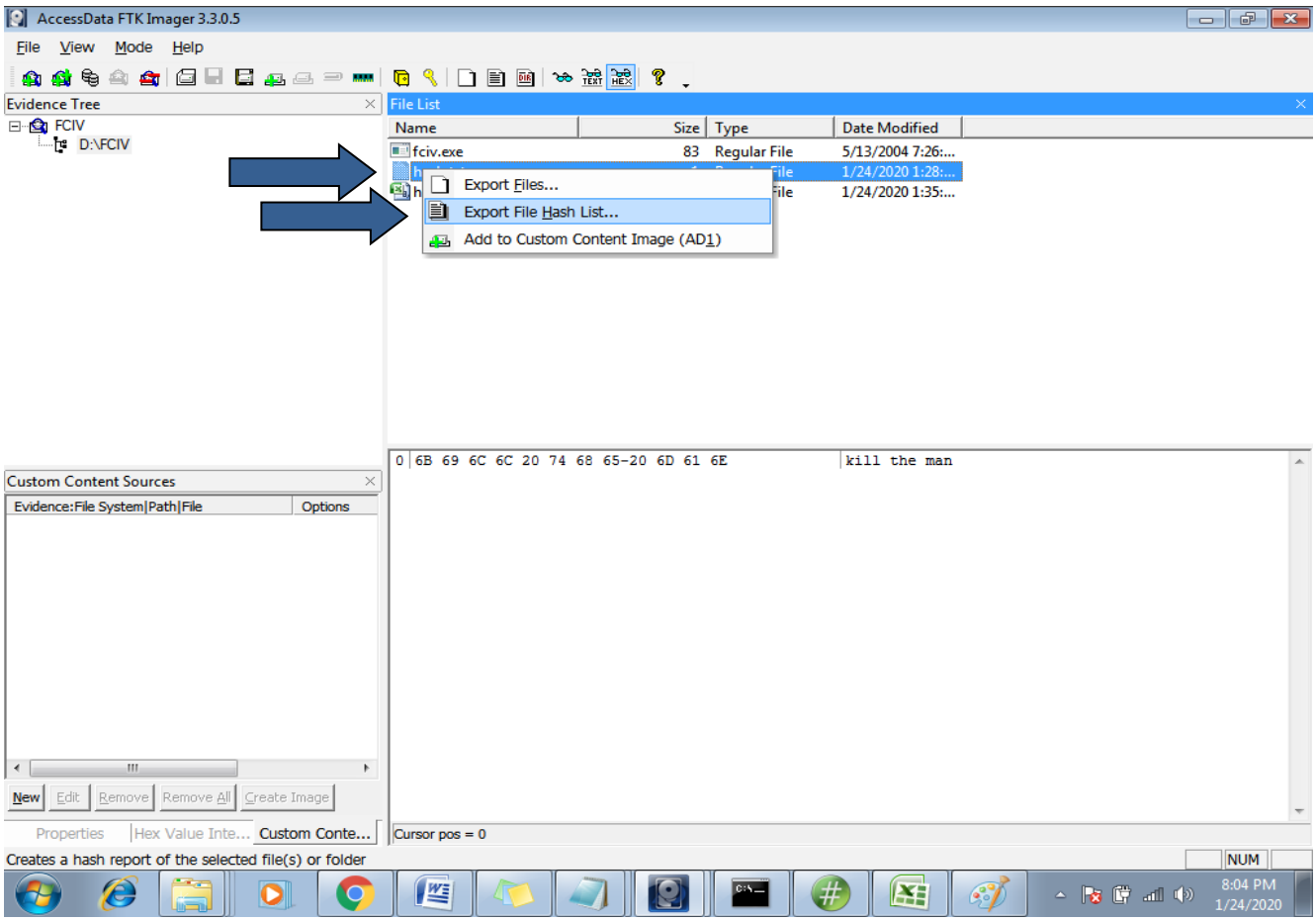
a) Open "FTK Imager" and click "**Add Evidence Item**" button and select "**Contents of a Folder**" inside radio button then click "**Next**" button.

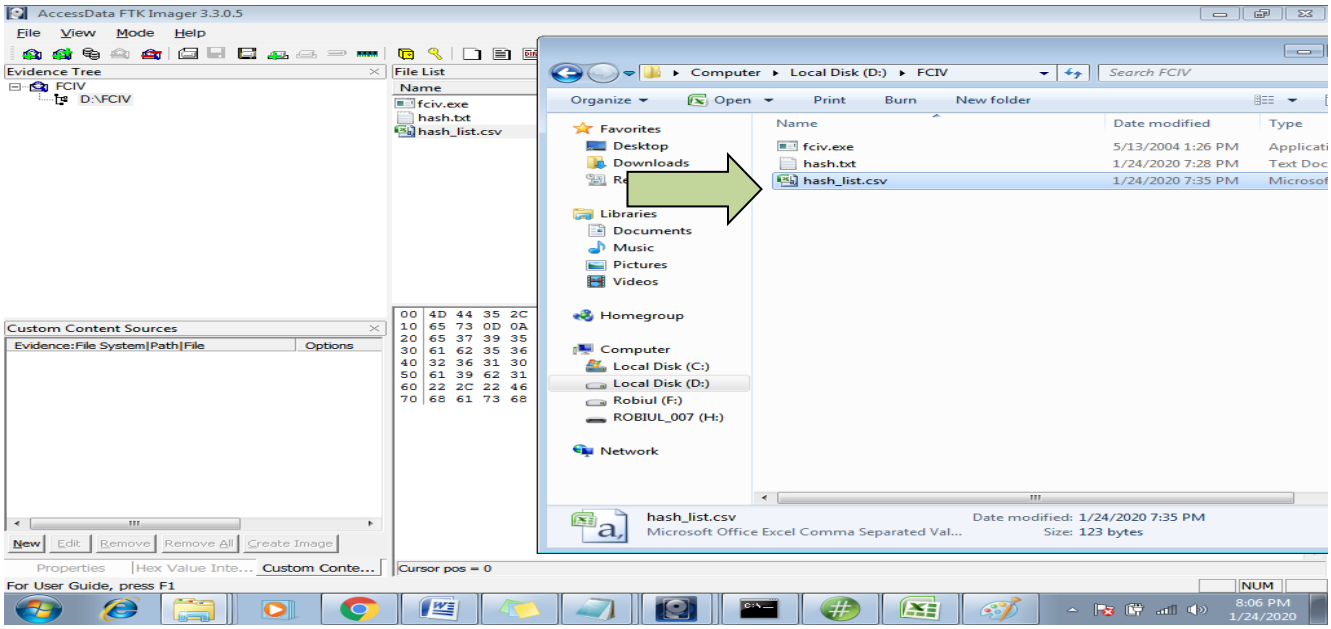


b) Browse **source path**: and click "**Finish**" button.

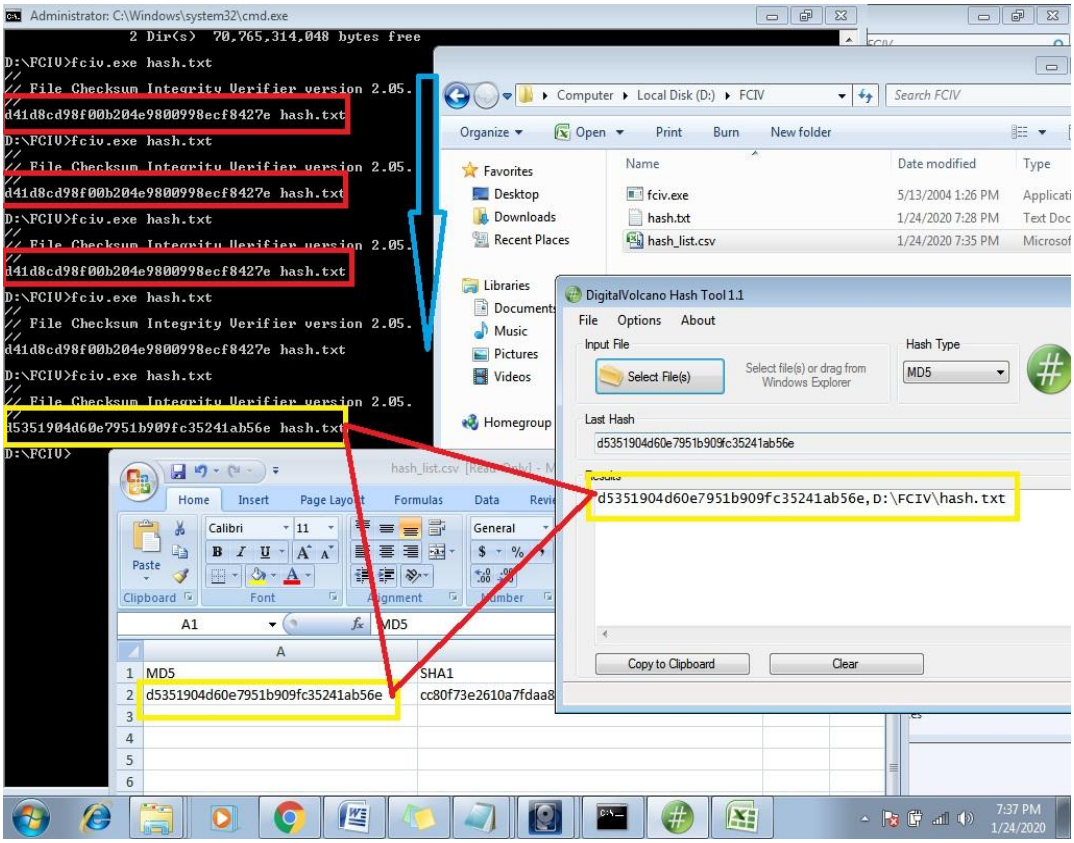


c) Export **file/files hash list/lists** and open **csv file/files**.



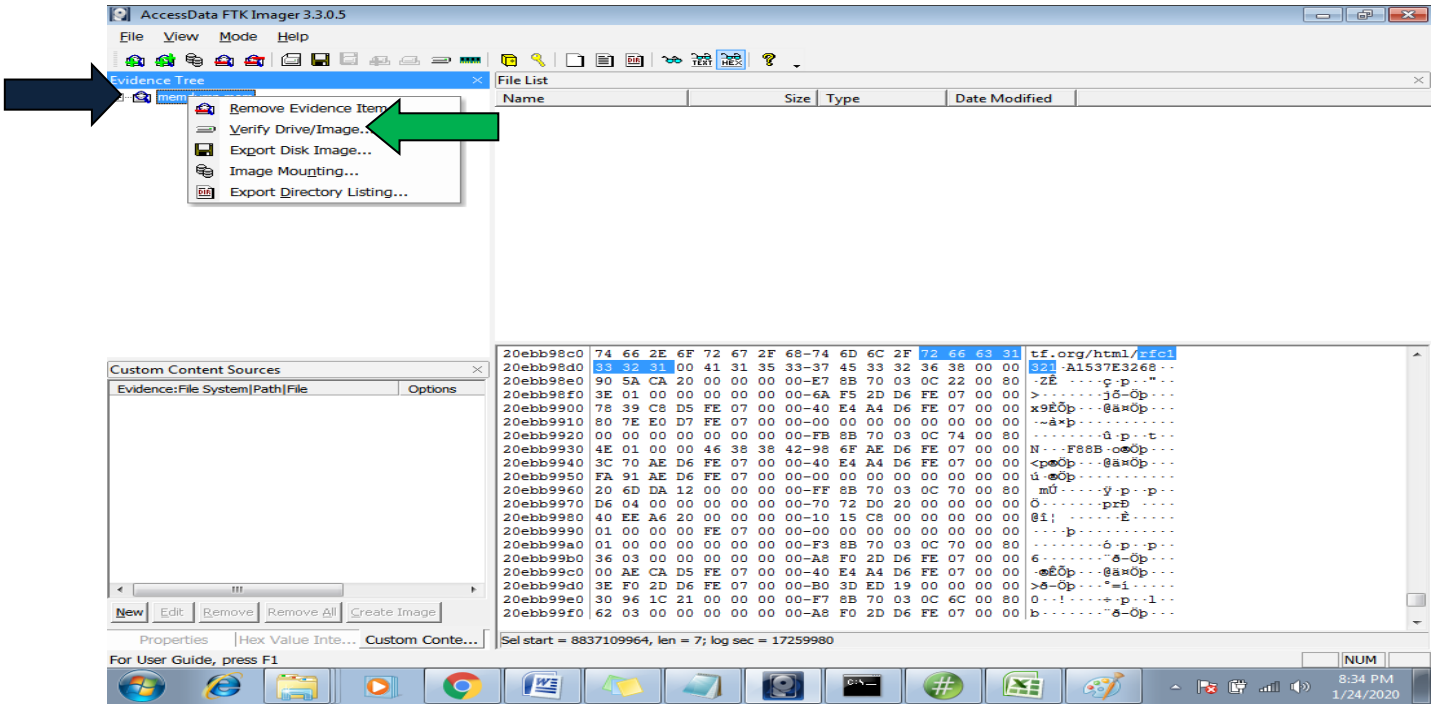


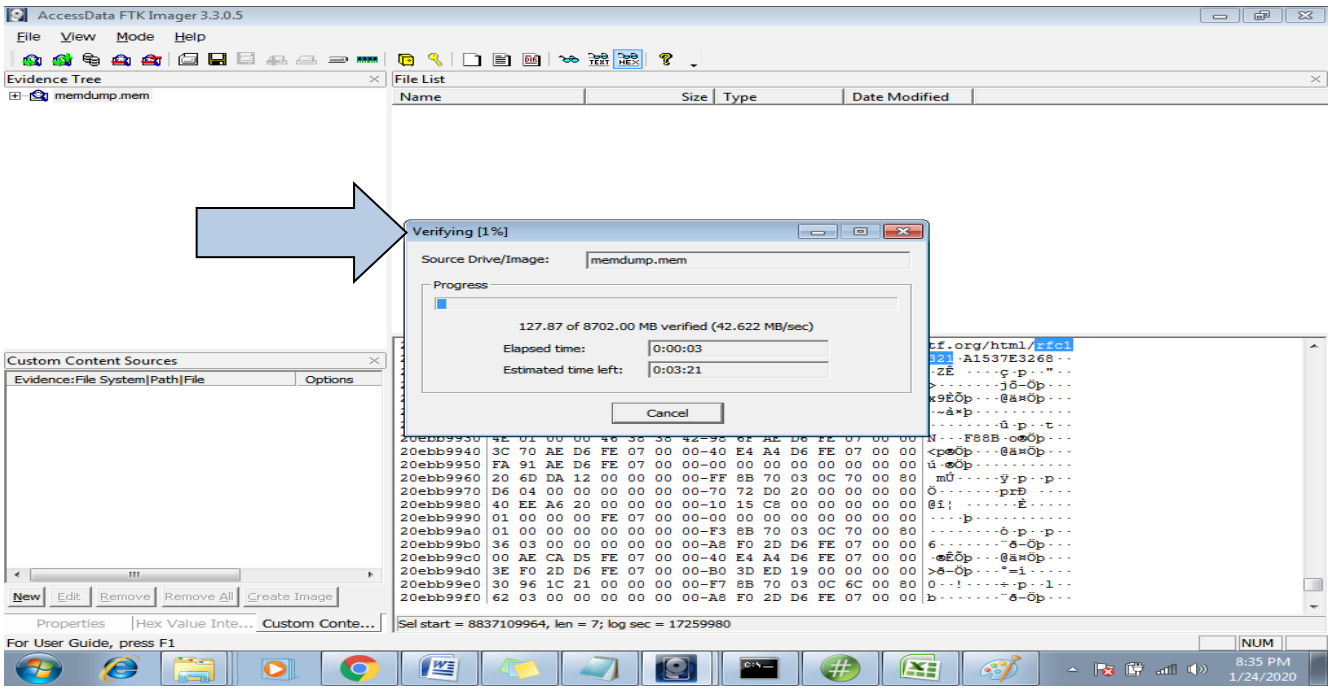
d) **Compare all generated hash values** by using different Tools. [Notice that: If the file's contents is same, hash value will be same]



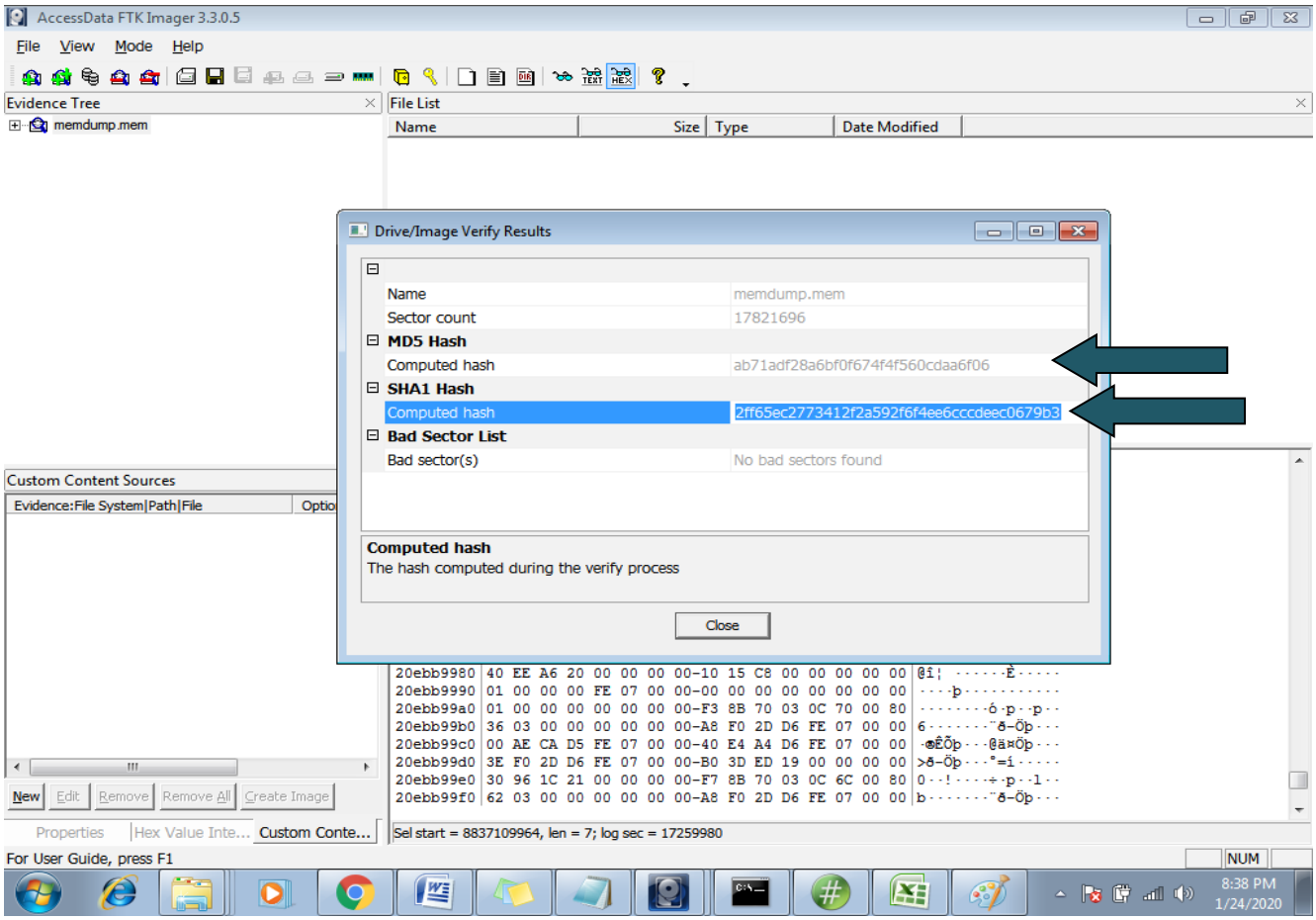
Verify memory image captured by FTK

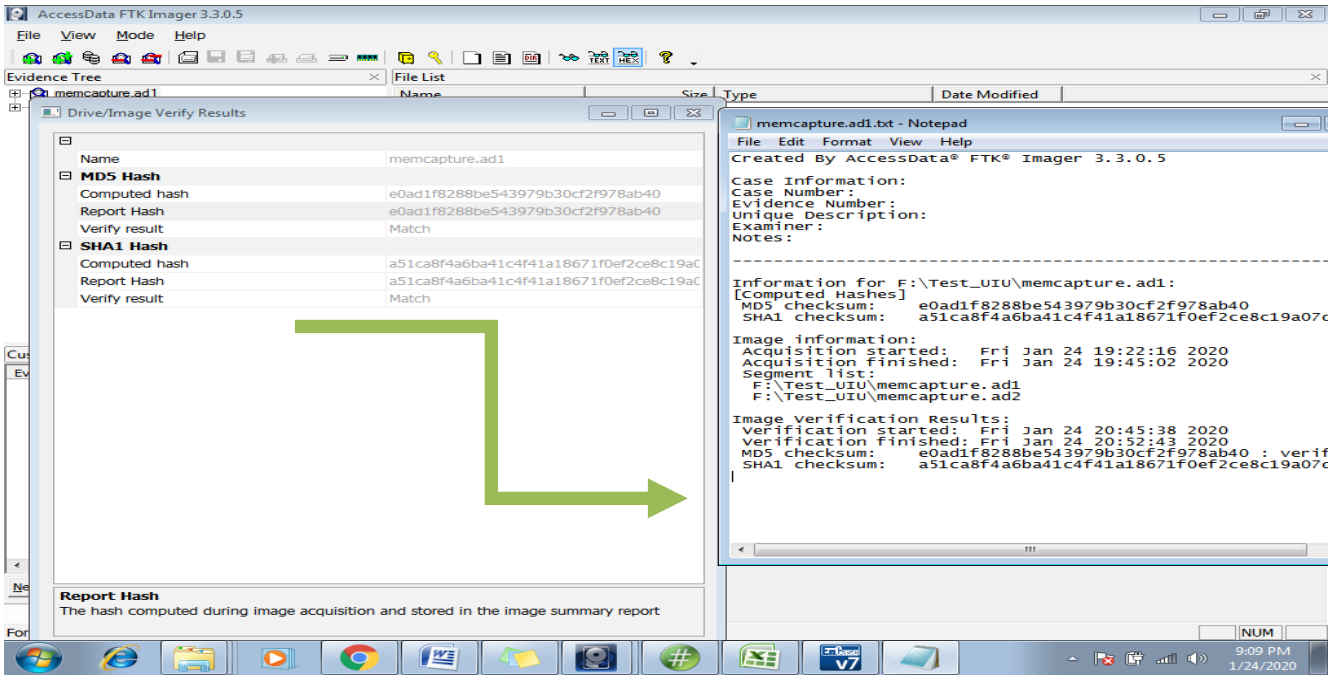
e) Select the **already opened image file** and right click on it and open "**Verify Drive/Image**".



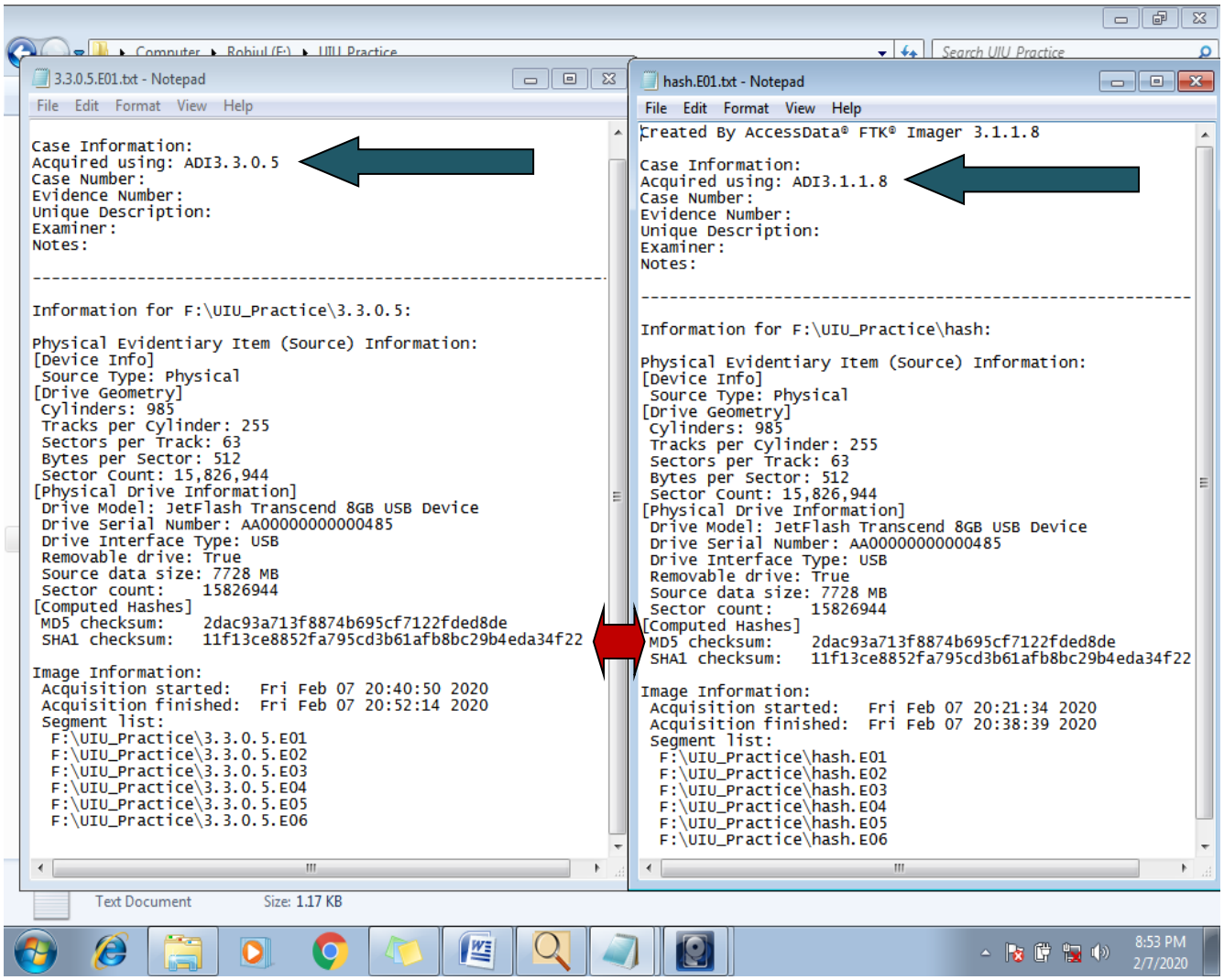
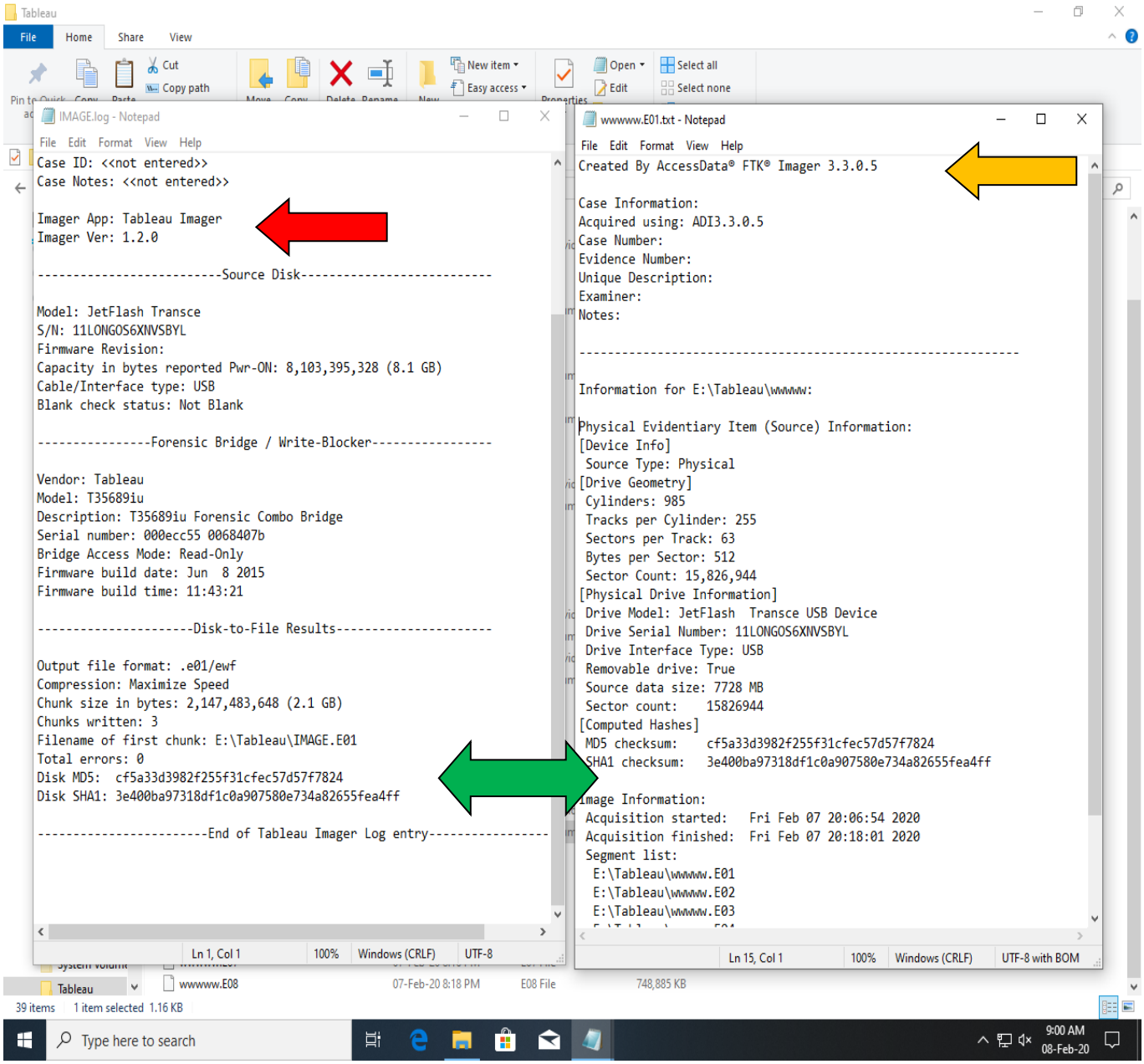


f) Compare between **verified hashes** and **pre-generated hashes**.





g) The Imager tool is same but versions are different, both generated hash values are same.



h) The Imager tools are different but both generated hash values are same.